Université Gustave Eiffel

L2 Informatique — Semestre 2

Algorithmique des Arbres

DM 1 — Vérification du statut « de recherche » pour un arbre binaire

Membres du groupe :

Raphaël **LIBÉRALE**

Néo **RISTIC**

Groupe de TD : **5**

29 mars 2026

0 Table des matières

1 Réalisation du DM et difficultés rencontrées **2**
1.1 Réalisation et organisation du travail 2
1.2 Difficultés rencontrées 2

2 Protocole expérimental **3**
2.1 Environnement d'exécution 3
2.2 Types d'arbres générés 3
2.3 Paramètres des mesures 4
2.4 Format des résultats 4
2.5 Script de compilation et d'exécution 4

3 Analyse des résultats **5**
3.1 Courbes obtenues 5
3.2 Analyse par méthode 5
3.2.1 Méthode naïve (`est_abr_naif`) 5
3.2.2 Méthode optimisée (`est_abr_definition`) 7
3.2.3 Méthode par parcours infixe (`est_abr_infixe`) 8
3.3 Comparaison des trois méthodes 9
3.3.1 Cas des arbres qui sont des ABR 9
3.3.2 Cas des arbres qui ne sont pas des ABR 9
3.3.3 Synthèse 9

Conclusion **10**

1 Réalisation du DM et difficultés rencontrées

1.1 Réalisation et organisation du travail

L'organisation du travail a été faite via un dépôt Github, lequel nous a permis de travailler en parallèle sur les différentes tâches du devoir et de synchroniser directement les avancées qu'on faisait. Quant au débuggage, l'outil `gdb` a été utilisé ; Cela a ainsi permis de découvrir des méthodes avancées de débuggage comme l'automatisation des breakpoints et des affichages pour aller plus vite à la source du problème.

La répartition du travail est la suivante :

Membre	Parties traitées
Raphaël LIBÉRALE	— Section 2.1.1 — Processus de construction des arbres sur un codage générique (`construit_quelconque`) — Section 2.1.2 — Génération des parcours préfixes des arbres complets, filiformes et quelconques. — Section 2.1.3 — Génération d'arbres ABR et non-ABR complets, filiformes et quelconques — 2.3 Rapport et discussions des résultats
Néo RISTIC	— Section 1.1 — Méthode naïve (`est_abr_naif`) — Section 1.2 — Méthode optimisée (`est_abr_definition`) — Section 1.3 — Méthode infixe (`est_abr_infixe`) — Section 2.2 — Protocole expérimental et script Python (`analyse_complexite.py`) — 2.3 Rapport et discussions des résultats

Table 1 – Répartition des tâches au sein du binôme.

1.2 Difficultés rencontrées

Raphaël LIBÉRALE : La génération des parcours préfixes depuis les parcours infixes a été technique car il fallait trouver le bon algorithme pour chaque forme d'arbre. Plus particulièrement, hormis le débuggage avec `gdb` assez long avec chacune des fonctions, la création du parcours préfixe d'un arbre filiforme a pris plusieurs essais d'algorithmes, dont les premiers étaient infructueux.

Ensuite, lors des générations d'arbres en fonction des formes, il fallait trouver comment implémenter les caractères de fin pour le codage de l'arbre (en l'occurrence `-1`) ; J'ai choisi

de le faire avant la création du parcours infixe sur les arbres complets (donc le tableau `infixe` est de taille $2n + 1$ avant l'appel de la fonction de parcours préfixe, où n est le nombre de nœuds). Pour les arbres filiformes et quelconques, cela se fait lors de la création du parcours infixe (donc `infixe` est de taille n avant l'appel de la fonction).

De plus, les tests étant automatisés, j'ai donc dû trouver un substitut des fonctions `pow`, `log2` et `floor` de la bibliothèque `<math.h>`, car je devais me passer de l'option de compilation `-lm`. J'ai donc trouvé sur internet une manière de faire en procédant avec les opérations bit à bit implémentées en `C` afin de reproduire ces calculs

Néo RISTIC : La première difficulté que j'ai rencontré a été la création de la fonction `est_abr_infixe` ou plus particulièrement sa fonction auxiliaire. Cette dernière n'est pas nécessairement longue mais m'a demandé une réelle compréhension récursive de l'algorithme. J'ai donc pris un crayon et une feuille et essayé de comprendre en dessinant des parcours infixe d'arbres binaires.

Mais le plus long a été la partie d'analyse des complexités. Notamment car les résultats graphiques obtenus paraissaient incorrects, ou du moins non représentatifs des complexités théoriques. Au final, nous nous sommes rendu compte que les résultats de notre analyse différaient des résultats théoriques car ils ne représentaient non pas le pire cas mais la moyenne de plusieurs arbres générés aléatoirement. Ainsi les complexités obtenues sont toutes inférieures aux complexités théoriques.

2 Protocole expérimental

Cette section décrit le protocole mis en place pour comparer expérimentalement les trois méthodes de vérification du statut ABR. L'objectif est qu'un lecteur puisse reproduire intégralement nos expériences.

2.1 Environnement d'exécution

- Système d'exploitation : Ubuntu 24.04.4
- Compilateur : `clang`, version 18.1.3
- Processeur : Intel(R) Core(TM) Ultra 7 155H
- Mémoire vive : 30 Gio de RAM (swap : 8 Gio)

2.2 Types d'arbres générés

Nous avons généré six types d'arbres binaires sans doublon, résumés dans le tableau ci-dessous.

Morphologie	**Type**	**Fonction de génération**
Presque complet	ABR	`ABR_presque_complet_alea`
Presque complet	Non-ABR	`non_ABR_presque_complet_alea`
Filiforme	ABR	`ABR_filiforme_alea`
Filiforme	Non-ABR	`non_ABR_filiforme_alea`
Quelconque	ABR	`ABR_quelconque_alea`
Quelconque	Non-ABR	`non_ABR_quelconque_alea`

TABLE 2 – Types d'arbres utilisés dans les expériences.

2.3 Paramètres des mesures

— Tailles testées : $n \in \{100, 600, 1100, \ldots, 9600\}$ noeuds

— Nombre de répétitions par point de mesure : 20

— Métrique mesurée : nombre de nœuds visités (type `long long`)

2.4 Format des résultats

Les résultats sont stockés dans le fichier `mesures.csv` dont le format de la première ligne est :

`Taille;Morphologie;Methode;Nb_visites;Temps`

2.5 Script de compilation et d'exécution

Grâce à la création d'un `makefile`, la compilation est hautement simplifiée et cela permet de reproduire les expériences suivantes avec les simples commandes suivantes :

```
make
./programme
python3 analyse_complexite.py
```

Listing 1 – Compilation et exécution

3 Analyse des résultats

3.1 Courbes obtenues

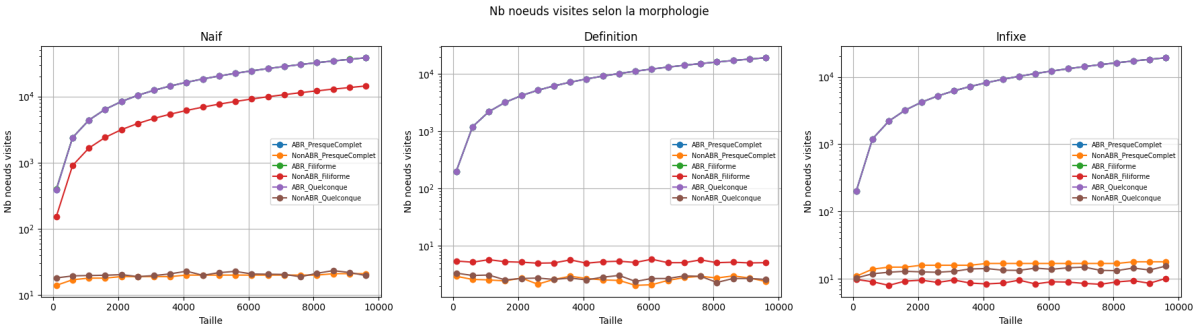

FIGURE 1 – Nombre de nœuds visités en fonction de la taille

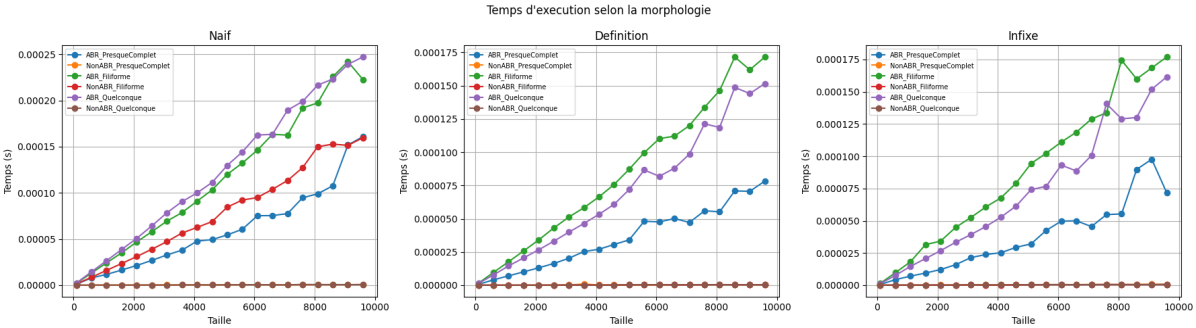

FIGURE 2 – Temps en fonction de la taille

3.2 Analyse par méthode

Nous poserons pour toute cette partie les notations suivantes :

— h : hauteur de l'arbre A ;

— n : nombre de noeuds de l'arbre A.

3.2.1 Méthode naïve (`est_abr_naif`)

Complexité théorique. Pour chaque nœud, la méthode naïve vérifie récursivement que les sous-arbres gauche et droit sont eux-mêmes des ABR, puis appelle `abr_max` sur le sous-arbre gauche et `abr_min` sur le sous-arbre droit. Ces deux fonctions descendent respectivement le long de la branche la plus à droite et la plus à gauche du sous-arbre, avec un coût proportionnel à la hauteur locale h_{local} du sous-arbre considéré. En sommant sur tous les nœuds, la complexité dans le pire cas est $\mathcal{O}(n \cdot h)$, où h est la hauteur de l'arbre. Dans le pire cas absolu (arbre filiforme), $h = n - 1$, ce qui donne une complexité en $\mathcal{O}(n^2)$.

Résultat clé

Complexité temporelle de `est_abr_naif` : $\mathcal{O}(n^2)$ dans le pire cas.

Validation expérimentale. Le pire cas théorique $\mathcal{O}(n^2)$ est atteint sur un arbre filiforme dont tous les nœuds n'ont que des fils gauches (ou que des fils droits), car `abr_max` doit alors descendre une chaîne de longueur k au niveau k, et on doit faire ceci n fois. En effet, on va d'abord visiter h nœuds, puis $h-1$, puis $h-2$, ..., puis 1, ce qui donne la somme

$$\sum_{k=1}^{n} k = \frac{n(n+1)}{2}$$

qui est bien quadratique.

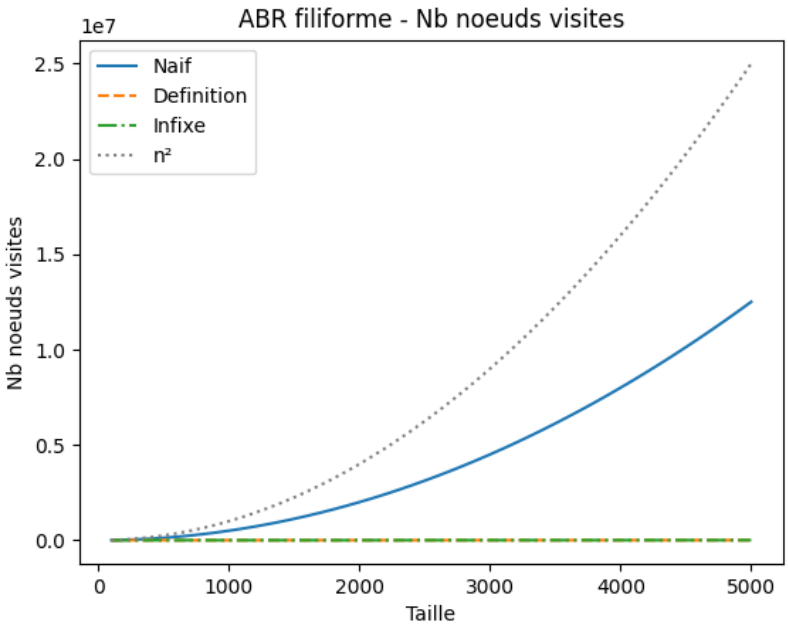

FIGURE 3 – Graphique du nombre de nœuds visités pour un filiforme de pire cas avec tous les enfants dans une seule direction.

Cependant, un tel arbre filiforme entièrement orienté dans un seul sens a une probabilité de $\frac{1}{2^{n-1}}$ d'être généré aléatoirement, soit $\frac{1}{2^{999}} \approx 0$ pour $n = 1000$. En pratique, un arbre filiforme aléatoire alterne les directions gauche et droite, de sorte que `abr_max` et `abr_min` coûtent $\mathcal{O}(1)$ en moyenne à chaque appel : la branche latérale opposée est quasi-inexistante.

Le nombre total de visites s'écrit alors :

$$T(n) = \underbrace{2n+1}_{\text{parcours récursif principal}} + \underbrace{2n}_{\text{appels à } \texttt{abr_max} \text{ et } \texttt{abr_min}\text{, coût amorti } \mathcal{O}(1)} = 4n+1$$

ce qui est bien linéaire, conformément à ce qu'on observe sur les courbes (pente constante ≈ 4 en échelle linéaire).

Pour un arbre presque complet, la hauteur vaut $h = \lfloor \log_2(n) \rfloor$. Pour un nœud situé à la profondeur d, son sous-arbre gauche a une hauteur de $h - d$. L'appel à `abr_max` sur ce sous-arbre doit descendre le long de la branche la plus à droite, ce qui coûte exactement $h - d$ visites. On peut donc calculer :

$$\sum_{d=0}^{h} 2^d \cdot (h-d) = 2^h \sum_{k=0}^{h} \frac{k}{2^k} \approx 2n \cdot 2 = 4n$$

car la série $\sum_{k=0}^{\infty} \frac{k}{2^k}$ converge vers 2. La complexité réelle est donc également $\Theta(n)$, bien que la borne théorique $\mathcal{O}(n \log n)$ soit formellement correcte mais très pessimiste.

Différences selon la morphologie. En pratique, les trois morphologies ABR (presque complet, filiforme, quelconque) produisent toutes une courbe de visites en $\approx 4n$, sans différence notable. Cela confirme que la constante multiplicative devant n est insensible à la forme de l'arbre, dès lors que les branches latérales restent courtes en moyenne.

La différence apparaît en revanche sur les *temps d'exécution* : `ABR_Filiforme` est nettement plus lent que `ABR_PresqueComplet` à nombre de visites égal, en raison de la profondeur de récursion $h = n$ qui génère n frames de pile consécutives et davantage de défauts de cache mémoire.

Pour les non-ABR, le comportement diverge fortement selon la morphologie. `NonABR_Filiforme` reste linéaire ($\approx 1.5n$ visites) car la violation est placée à une position aléatoire dans la chaîne : il faut en moyenne parcourir la moitié de l'arbre avant de la détecter. En revanche, `NonABR_PresqueComplet` et `NonABR_Quelconque` sont quasi-constants, la violation étant statistiquement proche de la racine dans notre méthode de génération.

3.2.2 Méthode optimisée (`est_abr_definition`)

Complexité théorique. La méthode optimisée propage des bornes $[\min, \max]$ à travers les appels récursifs, visitant chaque nœud exactement une fois (nœuds réels et sentinelles NULL incluses). Il n'y a aucun reparcours de sous-arbre. Sa complexité est donc $\mathcal{O}(n)$ dans tous les cas, avec une constante très proche de 2 (n nœuds réels + $n + 1$ appels sur NULL).

Résultat clé

Complexité temporelle de `est_abr_definition` : $\mathcal{O}(n)$ dans le pire cas.

Validation expérimentale. Les courbes montrent $2n + 1$ visites pour tous les ABR, quelle que soit la morphologie (les courbes sont superposées car la différences de quelques centaines de noeuds est trop faible pour être visible sur le graphique), ce qui correspond exactement au nombre de nœuds réels plus le nombre de feuilles `NULL` dans un arbre binaire à n nœuds. Cette valeur est indépendante de la forme de l'arbre, ce qui est validé par la superposition des courbes `ABR_PresqueComplet` et `ABR_Filiforme` en échelle logarithmique.

Pour les non-ABR, la détection est quasi-immédiate ($\mathcal{O}(1)$) dès que la violation est proche de la racine : la borne propagée est violée au premier nœud concerné et la récursion s'interrompt. Cela se traduit par des courbes plates au voisinage de 2 et 8 visites pour les trois morphologies.

Différences selon la morphologie. Pour les ABR, il n'y a aucune différence entre les morphologies : le nombre de visites est strictement $2n + 1$ dans tous les cas. Pour les non-ABR, la différence dépend uniquement de la *position de la première violation* dans l'ordre de parcours préfixe, qui est déterminée par la méthode de génération et non par la forme de l'arbre.

3.2.3 Méthode par parcours infixe (`est_abr_infixe`)

Complexité théorique. La méthode infixe réalise un parcours infixe de l'arbre en s'interrompant dès la première violation de l'ordre croissant. Dans le meilleur cas (violation au premier nœud visité en ordre infixe, c'est-à-dire au nœud le plus à gauche), sa complexité est $\mathcal{O}(h)$, où h est la hauteur de l'arbre, car il faut descendre jusqu'au premier nœud infixe avant de détecter l'anomalie. Dans le pire cas (arbre ABR), le parcours est intégralement réalisé et la complexité est $\mathcal{O}(n)$.

Résultat clé

Complexité temporelle de `est_abr_infixe` : $\mathcal{O}(n)$ dans le pire cas.

Validation expérimentale. Les courbes de visites pour les ABR sont superposées à celles de la méthode "Définition" ($2n + 1$ visites), ce qui confirme que le parcours infixe complet visite exactement autant de nœuds. Pour les non-ABR, les courbes sont quasi-constantes mais légèrement supérieures à celles de la méthode "Définition" : en effet, la méthode infixe doit d'abord descendre jusqu'au nœud le plus à gauche ($\mathcal{O}(h)$ visites) avant de commencer à comparer, alors que la méthode Définition peut détecter une violation dès la racine.

Différences selon la morphologie. Pour les ABR, le constat est identique à la méthode "Définition" : $2n + 1$ visites quelle que soit la morphologie. Pour les non-ABR sur arbre filiforme, le comportement est quasi-constant (≈ 9 à 10 visites observées) car la violation de l'ordre croissant est détectée très tôt dans le parcours infixe indépendamment de la position de la violation dans l'arbre.

3.3 Comparaison des trois méthodes

3.3.1 Cas des arbres qui sont des ABR

Sur les ABR, les trois méthodes visitent toutes $\Theta(n)$ nœuds, mais avec des constantes différentes. La méthode naïve visite environ $4n$ nœuds, contre $2n + 1$ pour les méthodes "Définition" et "Infixe". L'écart de constante (facteur ≈ 2) se retrouve directement sur les temps d'exécution. Les méthodes "Définition" et "Infixe" sont donc systématiquement plus efficaces sur les ABR, avec une performance quasi-identique entre elles.

3.3.2 Cas des arbres qui ne sont pas des ABR

L'interruption anticipée est le facteur déterminant. La méthode Définition est la plus efficace : elle peut détecter une violation dès la racine en $\mathcal{O}(1)$, car la borne $[\min, \max]$ est vérifiée avant toute récursion. La méthode Infixe doit d'abord descendre jusqu'au nœud le plus à gauche avant de commencer les comparaisons, ce qui lui coûte $\mathcal{O}(h)$ visites même si la violation se trouve proche de la racine — d'où les ≈ 9 à 10 visites observées.

La méthode naïve est la plus pénalisée sur les non-ABR filiformes : elle récurse d'abord jusqu'aux feuilles pour valider les sous-arbres, puis remonte pour comparer, de sorte qu'elle doit parcourir en moyenne la moitié de l'arbre avant de détecter la violation, soit $\approx 1.5n$ visites.

3.3.3 Synthèse

	Naïve	**Optimisée**	**Infixe**
Complexité pire cas	$\mathcal{O}(n^2)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Complexité pratique ABR	$\Theta(4n)$	$\Theta(2n)$	$\Theta(2n)$
Complexité meilleur cas	$\mathcal{O}(1)$	$\mathcal{O}(1)$	$\mathcal{O}(h)$
Avantage principal	Simple à implémenter	Optimale en tous cas	Découle naturellement de la propriété infixe
Inconvénient principal	Constante $\times 2$ sur les ABR	Moins intuitive à concevoir	Meilleur cas en $\mathcal{O}(h)$

Table 3 – Synthèse comparative des trois méthodes.

En conclusion, la méthode "Définition" est à recommander dans tous les cas : elle est optimale en $\mathcal{O}(n)$ sur les ABR, détecte les violations en $\mathcal{O}(1)$ dès la racine sur les non-ABR, et sa constante multiplicative (≈ 2) est la plus faible des trois. La méthode "Infixe" lui est équivalente sur les ABR mais légèrement moins efficace sur les non-ABR. La méthode naïve, bien que correcte, est à éviter en pratique en raison de son coût double sur les ABR et de sa sensibilité aux arbres filiformes.

3 Conclusion

Ce devoir nous a permis d'implémenter et de comparer trois méthodes de vérification du statut ABR, en confrontant leur complexité théorique à des mesures expérimentales.

On voit en particulier que les complexités théoriques sont en pratiques très différentes des complexités pratiques .

La méthode "Définition" s'impose finalement comme la plus efficace dans tous les cas : complexité $\mathcal{O}(n)$ garantie, détection en $\mathcal{O}(1)$ dès la racine pour les non-ABR, et constante multiplicative la plus faible. Ce DM illustre ainsi qu'une analyse algorithmique associée à une validation expérimentale est indispensable pour choisir la bonne implémentation en pratique.